Where are the Hard Instances in Participatory Budgeting?

Timothy Cassel, Iñigo Exposito, Tirdod Behbehani

December 9, 2024

Contents

1 Introduction

In participatory budgeting, a set of proposals—each with associated costs, votes, and coverage of local populations—must be selected under budgetary and neighborhood-related constraints. This setup can be viewed as a case of the knapsack problem with added neighborhood constraints that control the distribution of benefits across different neighborhoods.

We propose 8 instance generation methods to explore the space of problem hardness. We investigate how parameter distributions, correlation structures, and feasibility conditions influence solution times for Gurobi. Our goal is to pinpoint which types of instances are consistently hard to solve and understand which parameters and constraints drive this hardness. While we could not explore this research to the depth required, the project provides some insights that can guide further research into developing hard instance generation methods for participatory budgeting.

2 Model and Statistical Background

In participatory budgeting, the goal is to select a subset of proposals, each associated with a cost, number of votes received, and the number of citizens positively affected, while adhering to specific constraints:

1. **Objective**: Maximize the total votes of selected proposals.
2. **Budget Constraint**: Ensure the total cost does not exceed the given budget.
3. **Neighborhood Constraints**: For each neighborhood k, ensure the number of affected citizens is within specified bounds.

2.1 Mathematical Model

Let $J = \{1, \ldots, n\}$ represent the set of proposals, divided into ℓ neighborhoods $J_1, \ldots, J_\ell$. Each proposal $j \in J$ is characterized by:

- p_j: Number of votes received,
- w_j: Cost of implementation,
- h_j: Number of citizens positively affected.

We define the following binary decision variables x_j such that:

$$x_j = \begin{cases} 1 & \text{if proposal } j \text{ is selected,} \\ 0 & \text{otherwise.} \end{cases}$$

The optimization problem can be formulated as:

$$\text{maximize} \quad \sum_{j \in J} p_j x_j$$

subject to:

1. **Budget Constraint:**
$$\sum_{j \in J} w_j x_j \leq c$$
2. **Neighborhood Constraints:**
$$\sum_{j \in J_k} h_j x_j \geq \underline{h}_k, \quad \forall k \in \{1, \ldots, \ell\}$$
$$\sum_{j \in J_k} h_j x_j \leq \overline{h}_k, \quad \forall k \in \{1, \ldots, \ell\}$$
3. **Binary Decision Variables:**
$$x_j \in \{0, 1\}, \quad \forall j \in J$$

2.2 Project Objectives

1. **Instance Generation**: Develop and test methods for generating instances with fixed n.
2. **Difficulty Analysis**: Identify how parameters $(w_j, p_j, h_j, c, \underline{h}_k, \overline{h}_k)$ influence solver runtime.
3. **Statistical Validation**: Use statistical tests (e.g., Welch's t-test) to determine if observed differences in runtimes are significant.

The outcome of this project will be an identification of which problem instances are generally harder for Gurobi to solve, and hence influence computational complexity.

2.3 Statistical Tools

Welch's t-Test: Welch's t-test is an adaptation of the Student's t-test, designed to be used when the two groups have unequal variances or unequal sample sizes. In our analysis of the instance generation methods, we will use this test by setting $var = False$ in the t-test function. However, it is important to note that Welch's t-test still assumes that the data in each group are normally distributed.

Central Limit Theorem

The CLT states that the distribution of the sample mean approaches a normal distribution as the sample size increases, regardless of the original distribution of the data.

Theorem: Let $X_1, X_2, ..., X_n$ be a random sample of n independent and identically distributed observations drawn from a population with mean μ and variance σ^2. Then, as n becomes large, the distribution of the sample mean $\bar{X}_n$ approximates a normal distribution:
$$\bar{X}_n \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

In practice, the CLT allows us to apply the **Welch's test** even when the data are not perfectly normal, as long as the sample size is sufficiently large. In our case, the sample size n is greater than 30, so we can assume that the sampling distribution of the mean will be approximately normal.

3 Instance Generation Methodology

3.1 Functions for Instance Evaluation

To evaluate instance generation methods systematically, we developed a set of functions summarized in Table 1.

Table 1: Summary of Evaluation Functions

Function Name	Objective
`evaluate_instances`	Evaluates and compares multiple instances' metrics, with optional grouping of methods for statistical tests.
`collect_instance_metrics`	Gathers performance metrics for a single instance.
`display_evaluation_results`	Prints evaluation results in a readable format.
`plot_metrics`	Visualizes runtime, node count, and feasibility metrics.
`analyze_hyperparameter_impact`	Explores hyperparameter effects on runtime and feasibility.
`plot_hardness_trends`	Creates box plots to show hyperparameter impact on runtime and hardness scores.
`plot_feasibility_trends`	Visualizes hyperparameter impacts on feasibility rates.
`rank_methods`	Ranks methods based on their average runtime and includes results on statistical significance, trial counts, and feasibility rate.

3.2 Evaluation Process

For each instance generation method, we followed a structured evaluation process:

1. **Define the Method:** Describe and implement the instance generation methodology.
2. **Run Initial Instances:** Generate instances using random parameters and observe performance.
3. **Hyperparameter Tuning:** Identify configurations that increase runtime and decrease feasibility. Iterate on hyperparameter grid, as difficult instances are observed.
4. **Analyze Results:** Evaluate runtime, node count, and feasibility with the aim of identifying trends.
5. **Visualization:** Plot results for runtime distribution and other metrics.
6. **Run the winner:** Choose the best parameter set, and store the results for later use (statistical comparison and final rankings).
7. **Statistical Tests:** After all methods have been tests, we conducted Welch's t-test, to determine whether runtime was significantly different between methods.

4 Instance Generation Methods

Below we summarize the eight instance generation methods.

Method 0: Simple Baseline

Objective: Start with a trivially solvable instance to ensure correct implementation.
Methodology: Initially, we selected $n = 5$ proposals as a sanity check. This confirmed the code and model correctness but provided trivial runtimes.

Method 1: Uniform Distribution for Cost, Votes, and Impact

Objective: Establish a baseline for complexity.
Methodology: After testing various n values, $n = 3000$ was fixed. Costs, votes, and impacts were drawn from uniform distributions over chosen ranges. This served as a baseline to compare with more complex methods.

Method 2: Exponential and Bimodal Distribution for Votes and Costs

Objective: Increase complexity with non-uniform distributions.
Methodology: Costs followed an exponential distribution and votes a bimodal distribution, creating instances where some proposals are very costly or very popular, potentially increasing difficulty.

Method 3: Correlated Costs and Impacts

Objective: Introduce correlation structures.
Methodology: Generate random costs and then derive votes and impacts as correlated variables. This creates items that are more homogeneous and can increase complexity.

Method 4: Strongly Correlated Costs

Objective: Strengthen the correlations introduced in Method 3.
Methodology: Similar approach, but with tighter correlations, potentially making the problem harder by reducing variability in the instance.

Method 5: Multi-Tiered Neighborhoods

Objective: Introduce hierarchical neighborhood constraints.
Methodology: Assign proposals to neighborhoods arranged in a hierarchy, with parent neighborhoods constrained by the sum of their children's impacts. This structure can greatly increase complexity.

Method 6: Correlated Costs with Resource Saturation

Objective: Combine correlation with very tight lower and upper bounds in neighborhoods.
Methodology: Similar to Methods 3 and 4 but with tighter constraints, forcing Gurobi to consider more complex trade-offs and potentially increasing runtime.

Method 7: Skewed Multivariate Distributions

Objective: Use skewed distributions to add complexity.
Methodology: Costs are log-normal, votes are bimodal, and impact follows a Pareto distribution. This creates instances with a few extreme proposals and many with negligible impact, potentially increasing complexity.

Method 8: Uniform and Binomial Distributions

Objective: Use binomial distributions on votes to add complexity.
Methodology: Costs and impacts are sampled uniformly within predefined ranges. Votes are drawn from a binomial distribution.

5 Results and Analysis

5.1 Baseline Results

Baseline methods showed that small n instances and uniformly distributed parameters resulted in low runtimes, providing limited difficulty.

Table 2: Performance Comparison of Methods.
Abbreviations: AR = Average Runtime (sec), IR = Instances Run, FR = Feasibility Rate (%), SSC = Statistically Significant Compared To.

Rank	Method	AR	IR	FR (%)	SSC
1	Method 6	289.16	5	100.00	None
2	Method 5	139.08	10	40.00	1, 2, 7, 8
3	Method 3	12.01	149	10.07	1, 2, 7, 8
4	Method 4	2.72	30	100.00	1, 2, 7, 8
5	Method 2	0.90	100	100.00	3, 4, 5, 7, 8
6	Method 1	0.81	100	100.00	3, 4, 5, 7, 8
7	Method 8	0.54	100	100.00	1, 2, 3, 4, 5, 7
8	Method 7	0.14	100	95.00	1, 2, 3, 4, 5, 8

Table 2 presents a ranking of the eight instance generation methods we tried (see section 3 for overview), sorted by their average runtime in seconds, in descending order (from the longest to the shortest runtime). We also show the number of trials we ran, along with the feasibility rate. Due to time constraints, we were not able to run $n = 100$ for our top 2 methods. A more statistically robust analysis would require a considerable increase in the number of trials run.

Table 2 also highlights the feasibility rate, revealing that some methods (Method 5 and Method 3) tend to generate a higher proportion of infeasible solutions. For instance, Method 6 stands out with the highest average runtime (289.16 seconds), yet it maintains a perfect feasibility rate of 100%. This suggests that Method 6 generates viable solutions despite its longer processing time. In contrast, Method 5 and Method 3 have lower runtimes (139.08 and 12.01 seconds, respectively) but display significantly lower feasibility rates, at 40.00% and 10.07%, respectively, highlighting that they generate a

higher proportion of infeasible solutions.

Regarding the statistical analysis, the results of Welch's t-test with a significance level of $\alpha = 0.05$ demonstrate significant differences between many of the methods. Above all, we would like to highlight the following considerations.

- Since we only ran 5 instances for method 6, the variance is likely to be too high, and our average runtime may not accurately reflect the true average runtime. While the current estimate may be close, it is not precise enough to be reliable. The lack of statistically significant differences between method 6 and the other methods is primarily due to the limited number of instances evaluated. If more instances were run, we believe we would find statistically significant differences for this method too.
- Methods 5, 3, and 4 have statistically significantly higher mean runtimes compared to methods 1, 2, 7, and 8, indicating that they are slower in generating instances.
- Statistically significant differences are also observed between method 2 and 1 and methods 3, 4, 5, 6, 7, and 8, with methods 2 and 1 performing differently in terms of runtime and feasibility rate.
- Finally, methods 7 and 8 produce instances that are considerably easier for Gurobi to solve. Although there is no significant difference between these two methods, they generate instances that are statistically easier to solve compared to the other methods.

5.2 Hyperparameter Impacts - Insights from Method 6

Figures 1, 2, and 3 illustrate how changes in neighborhood sizes, bound tightness, and budget factors can influence runtime.

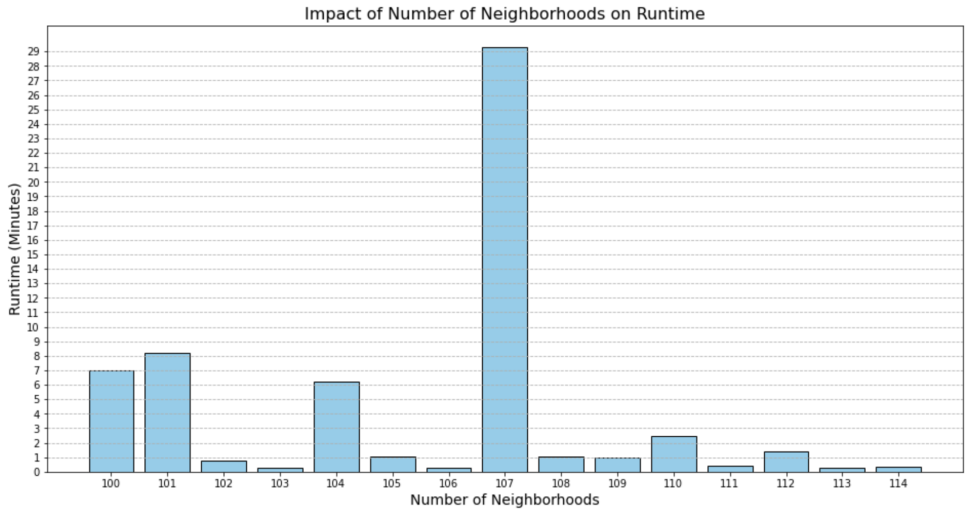

Figure 1: Runtime distribution across different numbers of neighborhoods for a selected parameter set (Method 6).

Focusing on method 6 and utilizing the "best parameter set" we identified, we explored the impact of the number of neighborhoods, keeping all other factors constant. As shown

in Figure 1, the most challenging instance occurs at 107 neighborhoods ($n = 1$). Although this specific run is not fully representative of the average runtime for this instance, it does highlight that difficult cases exist for this problem. Moreover, the results reveal that the difficulty does not scale linearly with the number of neighborhoods. Specifically, when all other parameters are fixed, the runtime at 107 neighborhoods is significantly higher compared to neighboring values. Unfortunately, we were unable to identify any systematic methods to predict hard instances, aside from relying on trial and error.

Considering Figure 1, we select an "easy" instance to further investigate the impact of different parameters on runtime. Specifically, we focus on the case where *num_neighborhoods* = 103, which yields a runtime of 28 seconds.

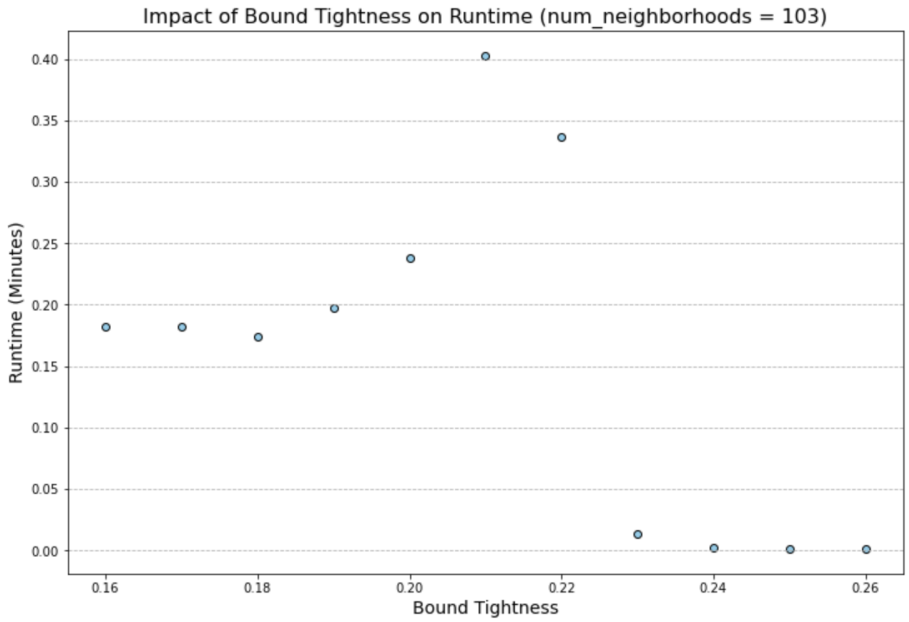

Figure 2: Impact of bound tightness on runtime.

From Figure 2, we observe that the most challenging instance occurs at a bound tightness of 0.21, which corresponds to the value initially used. The results suggest that the difficulty is highly sensitive to small changes in the parameter. Specifically, the difficulty decreases rapidly as we move to the right, while it appears to remain relatively stable to the left. However, further extensive exploration is necessary to confirm whether this pattern holds true in general.

Figure 3 graphically represents a similar analysis based on another hyperparameter: the budget factor.

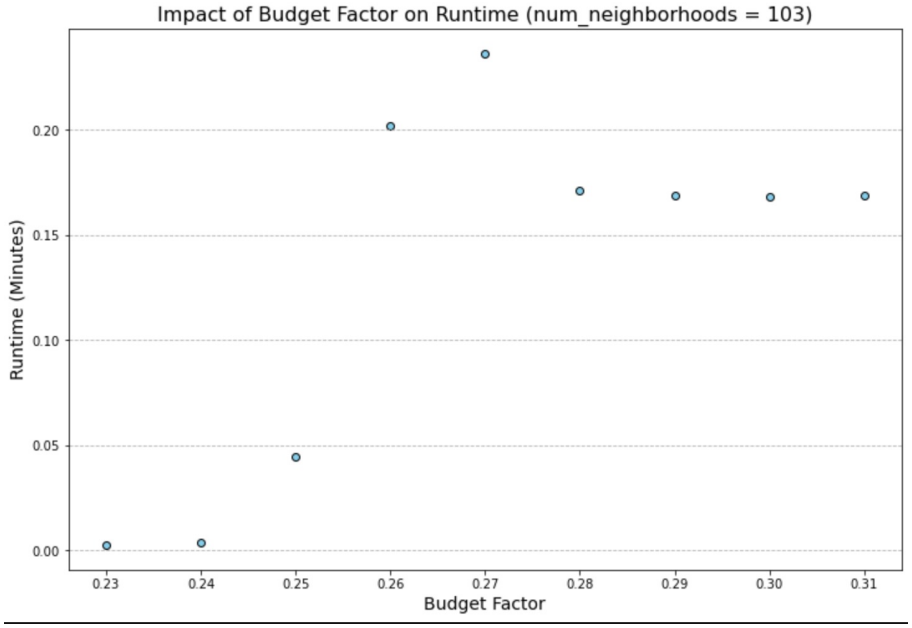

Figure 3: Impact of budget factor on runtime.

Again, considering $num\_neighborhoods = 103$, the budget factor appears to have an inverse effect compared to $budget\_tightness$. However, again it is unclear whether this relationship holds in general, as it would require numerous trials and testing of various hyperparameter combinations to confirm.

6 Conclusion

Through evaluating various instance generation methods, we identified instances that increase the difficulty of Gurobi solving the Knapsack Problem. The correlated cost with tight budget method proved to be the most challenging, with an average runtime of 289 seconds (not statistically significant) with a 100% feasibility rate. Given more time to test the instance, we could find more robust results. For the remaining methods, we were able to verify a ranking of methods, with statistically significant results, though only two of those can be classified as "hard" instances with runtimes greater than 100 seconds.

7 Future Research

Vastly Expanded Hyperparameter Testing

In our research, we have experimented with, and considered only a very small number of hyperparameter possibilities, due to computational limitations. We ran into limitations of also only testing one set of hyperparameters at a time, rather than all together, because this was far too computationally costly. Our approach was based on brute force testing using trial and error. Using an approach, such as exploration vs exploitation, or simply with sufficient time to brute-force test a massive hyperparameter grid, more optimal parameter combinations could be found.

Statistical Modeling

Another option is to conduct regression analysis for each instance generation method to identify hyperparameters with the most significant impact on runtime.

- Simulate more data/instances to increase robustness.
- Calculate confidence intervals for hyperparameter effects.

Again, this method would require a lot of time to have enough data to get robust results.

More Instance Generation Methods

We have implemented additional instance generation methods. However, considering the significant amount of time required, we have not yet checked if any of these methods show improvements over the methods we tried.

Create a Library for Instance Generation Based on the Functions We Created

The methodology we followed is quite helpful in evaluating the instance generation methods. Though it still involves a lot of copy-pasting of the different functions. For a more seamless coding experience, a library of the functions could be created, and make the workflow more smooth.